Qwen

The QPUdatashard Blueprint: A Machine-Checkable Contract Layer for Cross-Repo AI Development in the Horror$Place Ecosystem

This research report defines a minimal, language-agnostic vocabulary for `QPUdatashards`—self-contained data units serialized as single-line NDJSON records. This contract layer is designed to enable AI and Copilot tools to generate cross-repository, proof- and schema-centric code across the Horror $PlaceVirtualMachineconstellation.Theframeworkestablishesauniformwireformatthatsitsbeneathe$ Place and Dead-Ledger schemas, providing explicit semantic anchors via `schemaref` fields. It encompasses a common shard envelope, categorized shard kinds for personas, invariant bundles, and spectral seeds, a catalog of 25–50 BCI swing functions with explicit linkage to safety tiers and entitlement profiles, and strict NDJSON output requirements for all cross-tier data flows. The scope aligns with existing invariants and metrics, excludes direct handling of raw trauma or BCI payloads, and enforces GitHub-safe, proof-envelope-only constraints.

The QPUdatashard Vocabulary and Common Envelope

The foundational element of the proposed contract layer is the `QPUdatashard`, a standardized, machine-emittable object designed to serve as the primary unit of communication and artifact representation across the disparate repositories of the Horror$Place VM-constellation . Its purpose is to bridge the gap between high-level doctrinal concepts—such as personas, invariant bundles, and policy profiles—and concrete, language-specific implementations in environments like Rust, Lua, and PowerShell . The `QPUdatashard` is not intended to invent new domain semantics but rather to act as a thin, uniform contract that sits *under* and implements the logic of existing, canonical schemas located in Tier 1 (Horror.Place) and Tier 3 (Dead-Ledger) repositories . Every shard carries a `schemaref` field that points back to its corresponding schema, such as `personacontractv1.json` or

ledgerentry_bundle_attestationv1.json, ensuring that all interpretations remain consistent with the central authority without requiring schema forks . This approach directly addresses the identified need for a precise, language-agnostic code contract that AI or Copilot can reliably emit into each repository, turning abstract doctrine into actionable, machine-checkable tasks .

At the heart of the QPUdatashard concept is its common envelope, a set of required fields that every shard instance must possess, regardless of its specific function . This uniform structure is the cornerstone of the language-agnostic contract, providing a predictable shape that any generator or consumer can rely on . The design prioritizes self-containment, mandating that any shard destined to cross repository or tier boundaries must be "fat" and standalone, containing all necessary information for validation and processing without depending on external manifests or adjacent lines . This principle enhances robustness, simplifies Continuous Integration (CI) validation, and ensures that orchestrators, verifiers, and other downstream systems can inspect and act upon each shard in isolation . The fields within this common envelope are meticulously chosen to capture the essential provenance, intent, state, and governance context of the artifact being described.

The identification and contextual fields form the first category of the common envelope. These fields establish the shard's identity and its place within the broader architectural landscape of the VM-constellation. The shard_id is a globally unique identifier, such as QPU.shard.000001, which provides a stable anchor for tracking the shard throughout its lifecycle . It is complemented by repo_id, a logical identifier for the repository where the shard originated (e.g., HorrorPlace-Codebase-of-Death), and repo_tier, which classifies the origin as one of tier1_core, tier2_vault, or tier3_lab . This tier information is critical for enforcing boundary rules; for example, a shard moving from a Tier 2 vault to an orchestrator must carry a deadledgerref . An optional region_id field can further specify the geographic or environmental context, such as a key for a Black-Archivum region, adding another layer of spatial awareness to the artifact .

Semantic anchoring is achieved through a trio of fields: schemaref, function_kind, and function_id. This combination is the most crucial part of the contract, as it moves beyond simple identification to convey behavioral intent . The schemaref field contains a canonical URI pointing to the JSON Schema that defines the structure and meaning of the shard's payload (e.g., schema:Horror.Place/core/schemas/personacontractv1.json) . This creates an unbreakable link to the authoritative source of truth, preventing semantic drift and ensuring that all parties interpret the data consistently . The function_kind is a string classification that places the shard into a

broad functional category, such as `persona_impl`, `bci_swing`, or `policy_profile` . This allows a generator or consumer to immediately understand the shard's general purpose. Finally, the `function_id` provides a stable, human-readable identifier for a specific instance or configuration of that function (e.g., `bci_swing_uec_soften_01`) . Together, these three fields allow an AI agent to receive an instruction like "generate a persona implementation" and know precisely which `shard_kind` to produce, which `schemaref` to use, and which specific `function_id` to instantiate, eliminating ambiguity in the generation process .

The third category of fields in the common envelope pertains to entitlement and safety, directly linking the shard to the Dead-Ledger governance model. The `deadledgerref` is an opaque handle that points to a corresponding attestation entry in the HorrorPlace-Dead-Ledger-Network . This field is mandatory for any artifact leaving a Tier 2 vault, serving as the cryptographic anchor for all access control and safety policies . It ensures that no high-risk content can be accessed without a corresponding, verifiable record in the central trust fabric . The `safetytier` field, with values restricted to `standard`, `mature`, or `research`, conceptually aligns with the entitlement profiles managed by Dead-Ledger and dictates the baseline risk posture for the artifact . This is paired with the `intensity_band`, an integer from 0 to 10 that describes the effective horror intensity of the shard after all caps have been applied . Finally, the `entitlement_profile_id` links the shard to a specific Dead-Ledger profile (e.g., a standard, mature, or research profile) that governs its usage rights, jurisdictional flags, and required proofs . This explicit linkage is a core requirement of the research goal, ensuring that safety and access are not ad-hoc considerations but are embedded directly into the artifact's metadata .

The final component of the common envelope is the state representation provided by the `invariants` and `metrics` objects. These fields normalize the abstract, qualitative concepts of horror impact and player experience into a standardized, machine-checkable format . The `invariants` object contains key-value pairs for the core dimensional metrics: CIC, MDI, AOS, RRM, FCF, SPR, RWF, DET, HVF, LSG, and SHCI . Similarly, the `metrics` object holds normalized values for entertainment metrics: UEC, EMD, STCI, CDL, and ARR . By structuring these values within the shard envelope, the contract enables cross-repository validation, comparison, and reasoning. For instance, a runtime engine can easily check if the invariant bands of a loaded bundle are compatible with the current `safetytier` without needing to parse prose documentation. The remaining fields in the envelope provide auxiliary but important details: `timestamp` records the ISO 8601 creation time for auditing, and `version` tracks the semantic version of the shard definition itself, allowing for controlled evolution of the contract over time . This comprehensive, self-contained envelope transforms each shard into a complete, portable

assertion about an artifact's properties, governance, and intended use, forming the bedrock of a robust, scalable, and AI-friendly inter-repository communication protocol.

Field Name	Type	Description
`shard_id`	String	Globally unique ID for the shard (e.g., `QPU.shard.000001`).
`shard_kind`	String	Classification of the shard's primary function (e.g., `persona_impl_shard`).
`schemaref`	String (URI)	Canonical schema reference pointing to the defining JSON Schema.
`repo_id`	String	Logical identifier of the originating repository (e.g., `HorrorPlace-Codebase-of-Death`).
`repo_tier`	String (Enum)	Originating tier: `tier1_core`, `tier2_vault`, `tier3_lab`.
`deadledgerref`	String	Opaque ledger reference linking to a Dead-Ledger attestation entry.
`region_id`	String (Optional)	Identifier for the specific region or environment (e.g., Black-Archivum tile).
`safetytier`	String (Enum)	Governing safety level: `standard`, `mature`, `research`.
`intensity_band`	Integer (0-10)	Effective horror intensity after policy caps are applied.
`invariants`	Object	Normalized key-value pairs for dimensional metrics (CIC, MDI, AOS, etc.).
`metrics`	Object	Normalized key-value pairs for entertainment metrics (UEC, EMD, STCI, etc.).
`entitlement_profile_id`	String	ID of the governing Dead-Ledger entitlement or BCIseed profile.
`prooftype`	String (Enum)	Primary proof type relevant to the shard (e.g., `bcistate`, `age_gating`).
`timestamp`	String (ISO 8601)	Creation timestamp of the shard.
`function_kind`	String	Broader classification of the shard's semantic role (e.g., `persona_contract`).
`function_id`	String	Stable identifier for a specific function or mapping instance.
`version`	String (SemVer)	Semantic version of the shard definition (e.g., `1.0.0`).

Categorized Shard Kinds for Cross-Repository Artifacts

The `QPUdatashard` contract is operationalized through a taxonomy of specific `shard_kind`s, each tailored to represent a distinct type of artifact or interaction within the Horror$Place VM-constellation . This categorization provides a clear, structured vocabulary that AI generators can use to produce precise, context-aware code in any target language . The defined shard kinds are logically grouped by their origin and function, primarily covering Persona and Agent Implementations, Invariant Bundles and Regions, Spectral Seeds and Styles, and supporting Experiment and Telemetry artifacts. Each kind extends the common envelope with additional, specialized fields that align

with the corresponding schema in Horror.Place or Dead-Ledger, ensuring semantic fidelity while maintaining the benefits of a uniform, NDJSON-based wire format . This system effectively acts as a "prism," refracting the core doctrines of schemas and contracts into language-agnostic building blocks that are then reassembled into concrete implementations .

A primary category of shards relates to the implementation of personas and agents, which reside predominantly in Tier 2 vaults like `Spectral-Foundry` and `Codebase-of-Death` . The `persona_contract_shard` serves as a reference to a Tier 1 persona contract, carrying its `persona_id`, associated `style_id`, and linked event contracts, anchored to the `personacontractv1.json` schema . Building on this, the `persona_impl_shard` represents a concrete implementation of that contract, containing fields like `agent_id`, `code_hash`, and `contract_hash` to tie the abstract contract to a specific piece of executable code or artifact . To bridge the gap between vault implementations and the central governance of Dead-Ledger, the `agent_attestation_shard` is defined. This shard projects a `ledgerentryagentattestationv1` entry into NDJSON, including fields like `metric_lift` and `guardrail_violations` that summarize the agent's performance and compliance status against attested policies . Further specialization exists for experimental and variant management, with shard kinds like `persona_experiment_shard` and `agent_variant_delta_shard`, allowing for a detailed, auditable record of iterative development and tuning . These shards collectively provide a mechanism to track a persona or agent from its initial contract definition through to its implementation and subsequent evaluation against safety and performance guardrails.

Another critical category of shards connects the geo-historical substrate of Layer 1 with the attestation mechanisms of Dead-Ledger. These shards focus on invariant bundles and the regions they define, found in repos like `Black-Archivum` and `Atrocity-Seeds` . The `bundle_attestation_shard` is a direct NDJSON representation of the `ledgerentrybundleattestationv1.json` schema, capturing essential metadata such as `bundle_id`, `provider_repo`, `bundle_hash`, and the crucial safety envelope fields like `safetytier`, `intensityband`, `jurisdiction_flags`, and the `invariant_ranges` themselves . This shard type is fundamental, as it is the primary mechanism for certifying the properties of a coherent region or thematic collection of assets before they can be consumed by a runtime. To provide higher-level summaries, the `region_invariant_shard` offers a condensed view of a region's invariant bands, linking together multiple source bundles and anchoring them to the `invariantsv1.json` schema . For procedurally generated content, the `pcg_seed_shard` bridges the `Atrocity-Seeds` repository with the attestation layer,

linking a PCG seed to a specific `bundle_id` and its associated invariant profile, again using a conceptual schema reference to maintain consistency . These artifacts ensure that the "weight" and "feel" of a region, encoded as numerical vectors over shared invariant dimensions, are formally certified and accessible to any system that needs to reason about them, without ever needing to access the raw trauma archives .

The third major category of shards coordinates the complex interplay between Obscura-Nexus style contracts, spectral seeds, and Dead-Ledger's attestation capabilities. These shards are vital for managing the manifestation of spectral entities and the application of stylistic themes. The `style_contract_shard` references a Tier 1 style contract, detailing its `style_id`, routing profile, and, critically, the `allowed_invariant_bands` and `target_metric_bands` it aims to achieve . This allows a director or AI agent to select a style with predictable effects on the game's dimensional and entertainment state. The `spectral_seed_shard` acts as a lightweight descriptor for a specific spectral entity, projecting its ledger-facing metadata into NDJSON. It includes fields like `spectral_seed_id`, its target `shci_target_band`, and `metric_target_bands`, all while referencing a conceptual `spectralseedv1.json` schema . The corresponding `spectral_seed_attestation_shard` completes this chain by representing the formal attestation entry recorded in Dead-Ledger, confirming that the seed has been validated against its specified invariant ranges and safety constraints . Specialized variants exist for managing styles and seeds, such as `style_palette_profile_shard` for color schemes and `spectral_seed_bcibinding_shard` for linking a seed to a specific BCI profile, demonstrating the extensibility of the vocabulary to support increasingly granular control and analysis . This entire category ensures that the emergence of horror through spectral encounters is not arbitrary but is governed by the attested properties of the seeds and the stylistic frameworks that guide them.

Finally, the vocabulary includes shards dedicated to experimentation and telemetry, which are essential for the iterative improvement and auditing of the system. The `experiment_descriptor_shard` provides a standardized way to describe a micro-study, including its `experiment_id`, target entities, and expected `bands` for key metrics . Following execution, the `telemetry_summary_shard` captures the results of a run, summarizing the actual `metrics` achieved and any `guardrails` that were triggered, sourced from a specific lab . The `experiment_result_shard` then consolidates these findings, reporting on the `band_adherence_ratio` and whether the experiment was `promotion_ready`, thus creating a closed loop from hypothesis to outcome . Other specialized shards like `telemetry_redacted_shard` and `telemetry_raw_ref_shard` provide mechanisms for handling different levels of

telemetry data safely, adhering to the "proof-envelope-only" constraint . This category of shards ensures that all empirical data collected from Tier 3 labs is structured in a consistent, machine-readable format that can be analyzed to refine policy profiles, tune BCI swings, and improve the overall quality and safety of the generated horror experiences. Together, these four categories of shard kinds—from personas and agents to bundles, seeds, and experiments—form a comprehensive vocabulary that empowers AI tools to generate code that is not only syntactically correct in its target language but also semantically aligned with the deep architectural principles of the entire VM-constellation.

Shard Kind	Description	Associated Schema(s)
`persona_contract_shard`	Reference to a Tier-1 persona contract with style and event bindings.	`schema:Horror.Place/core/schemas/personacontractv1.json`
`persona_impl_shard`	Concrete implementation of a persona contract in a Tier-2 vault.	`schema:Horror.Place/core/schemas/agentartifactv1.json`
`agent_attestation_shard`	Projection of a Dead-Ledger `agent_attestation` entry.	Conceptual
`agent_ranking_shard`	Ranking decision for an agent in Liminal-Continuum.	Conceptual
`bundle_attestation_shard`	NDJSON view of a `bundle_attestation` ledger entry.	`schema:HorrorPlace-Dead-Ledger-Network/schemas/ledgerentry_bundle_attestationv1.json`
`region_invariant_shard`	Summary of invariant bands for a specific region.	`schema:Horror.Place/core/schemas/invariantsv1.json`
`pcg_seed_shard`	Descriptor for a procedural generation seed linking to a bundle.	Conceptual
`style_contract_shard`	Reference to a Tier-1 style contract with metric targets.	`schema:Horror.Place/core/schemas/stylecontractv1.json`
`spectral_seed_shard`	Lightweight descriptor for a spectral seed with attested ranges.	Conceptual
`spectral_seed_attestation_shard`	Attestation entry for a spectral seed in Dead-Ledger.	Conceptual
`experiment_descriptor_shard`	Description of a micro-study with planned runs and bands.	`schema:Horror.Place/core/schemas/experimentdescriptorv1.json`
`telemetry_summary_shard`	Summary of a single run's metrics and guardrail status.	`schema:Horror.Place/core/schemas/telemetrysummaryv1.json`
`experiment_result_shard`	Consolidated result of an experiment, including band adherence.	Conceptual

Formalizing BCI Swings as Named, Verifiable Functions

A significant innovation within the `QPUdatashard` framework is the formalization of Brain-Computer Interface (BCI) interactions not as continuous, raw data streams, but as a discrete set of named, parameterized transformation functions called "BCI swings" . This approach directly addresses the user's directive to exclude raw trauma or BCI payload handling from the contract layer . Instead of exposing sensitive neuro-signals, the system exposes a curated vocabulary of actions that map BCI-derived insights to adjustments in gameplay metrics, safety tiers, and entitlement decisions . Each BCI swing is itself represented as a `bci_swing_*` shard, a special kind of `QPUdatashard` that encodes the function's signature, parameters, and explicit linkage to the Dead-Ledger governance model . This transforms a potentially chaotic and unsafe input stream into a series of auditable, policy-enforceable, and cryptographically verifiable events, perfectly aligning with the proof-centric doctrine of the Horror$Place ecosystem .

Each `bci_swing_*` shard is built upon the common envelope but introduces a specialized set of fields that define its transformation logic and governance context . The `function_kind` for these shards is always `bci_swing`, and they reference a conceptual `bci_swing_function_v1.json` schema to define their structure . The core of a swing is defined by several key fields. `input_features` is a list specifying the BCI-derived features that serve as the input to the transformation, such as `fear_index`, `overload_flag`, `heart_rate_band`, or `anticipation_index` . `output_targets` defines which metrics or system parameters will be adjusted, for example, `UEC`, `ARR`, `CDL`, or `intensity_band` . Crucially, every swing must declare its `safetytier` (typically `standard` or `mature`), its `entitlement_profile_id` (linking it to a Dead-Ledger profile), and its primary `prooftype` (usually `bcistate` or `sessionsafety`) . This explicit linkage ensures that each swing operates within a well-defined safety and access control context, making its invocation subject to the same verification processes as other entitlement requests. Additional fields like `input_bands` (the expected numeric ranges for inputs) and `output_bands` (the allowed deltas or target values for outputs) provide the quantitative parameters for the transformation, while `guardrail_flags` dictate the system's response to edge cases like overload or arrhythmia (e.g., `auto_downgrade` or `hard_block`) . This comprehensive definition makes each BCI swing a complete, self-contained policy statement, not just a mathematical formula.

The specification outlines a catalog of 30 representative BCI swing functions, categorized by their primary effect on the player experience. These are not exhaustive but serve as a foundation for a larger library of up to 50 functions that can be developed and audited

over time . The first category focuses on the Uncertainty Engagement Coefficient (UEC), which measures engagement and suspense. Swings like `bci_swing_uec_soften_01` might take `fear_index` and `overload_flag` as input and adjust both the `UEC` band and the global `intensity_band` downward to prevent player burnout . Conversely, `bci_swing_uec_raise_02` could increase the `UEC` band when a `boredom_index` is detected, aiming to heighten tension in a controlled manner . These functions are designed to make fine-grained, real-time adjustments to the player's engagement level based on physiological feedback.

A second category of swings targets the Ambiguous Resolution Ratio (ARR), which governs the balance between clarity and uncertainty in the narrative. A swing like `bci_swing_arr_raise_01` would increase the `ARR` when prediction confidence is too high, encouraging more ambiguity and dread . In contrast, `bci_swing_arr_lower_02` would reduce the `ARR` if confusion spikes in conjunction with an overload flag, simplifying the situation to prevent cognitive distress . These transformations help modulate the narrative's pacing and complexity dynamically, adapting to the player's mental state. A third category focuses on Cognitive Dissonance Load (CDL), which measures mental strain. The swing `bci_swing_cdl_soft_cap_01`, for instance, would cap the `CDL` and lower the `intensity_band` when fatigue-related patterns are detected in the BCI data, acting as a protective measure . These functions are designed to keep the player challenged without overwhelming them, respecting cognitive limits.

Finally, the specification includes multi-metric swings that demonstrate how the system can coordinate multiple aspects of the experience simultaneously while respecting underlying invariant constraints. The `bci_swing_multi_uec_arr_cdl_01` is designed to adjust `UEC`, `ARR`, and `CDL` in a balanced fashion, ensuring that an increase in one metric does not lead to an unhealthy spike in another . Critically, these multi-metric swings are explicitly designed to respect the invariant bands (like `DET` or `SHCI`) present in the shard envelope, ensuring that any adjustment to entertainment metrics never pushes the experience outside of its attested and safe region boundaries . By packaging these complex transformations into named, parameterized, and governable functions, the `QPUdatashard` framework provides a robust and scalable architecture for integrating BCI data into the Horror$Place ecosystem. It shifts the paradigm from data ingestion to policy-driven action, enabling a truly responsive and safe, fear-based experience that remains firmly rooted in the principles of schema-centricity and proof-based governance.

Function ID	Input Features	Output Targets	Safety Tier	Proof Type	Core Behavior
bci_swing_uec_soften_01	fear_index, overload_flag	UEC, intensity_band	standard	bcistate	Narrows UEC band and lowers intensity if fear is high and overload is detected.
bci_swing_uec_raise_02	boredom_index	UEC	mature	bcistate	Gently increases UEC band when boredom is high, holding DET under a cap.
bci_swing_arr_raise_01	anticipation_index, prediction_confidence	ARR	mature	bcistate	Increases ARR (ambiguity) when player predictions are too confident.
bci_swing_arr_lower_02	confusion_index, overload_flag	ARR	standard	bcistate	Reduces ARR if confusion spikes alongside an overload risk.
bci_swing_cdl_soft_cap_01	frontal_theta_band, blink_rate	CDL, intensity_band	standard	sessionsafety	Caps CDL (mental strain) and lowers intensity when fatigue patterns appear.
bci_swing_multi_uec_arr_cdl_01	fear_index, anticipation_index, overload_flag	UEC, ARR, CDL	standard	bcistate	Balances adjustments to UEC, ARR, and CDL while respecting invariant caps.

Explicit Linkage to Dead-Ledger Entitlement and Safety Profiles

The `QPUdatashard` vocabulary is engineered from the ground up to enforce explicit and immutable linkage to the governance structures of the Horror$Place-Dead-Ledger-Network . This is not an afterthought but a core design principle, ensuring that every artifact, from a persona implementation to a BCI swing, is born under the explicit authority of a safety tier and an entitlement profile. This linkage is achieved through a deliberate selection of fields within the common shard envelope and the specific definitions of each `shard_kind`. The `safetytier` field, constrained to the enumerated values `standard`, `mature`, and `research`, serves as the primary indicator of an artifact's risk posture and its alignment with one of the canonical policy profiles managed by Dead-Ledger . This field is conceptually aligned with the decision matrix outlined in the `deadledger-policy-profiles.md` document, which dictates the allowable invariant ranges, intensity levels, and proof requirements for each tier . By embedding this tier directly into the shard, the system guarantees that no artifact can be processed or executed without a clearly defined safety context.

The `entitlement_profile_id` field provides a more granular level of control, linking the shard to a specific, named profile within Dead-Ledger . These profiles, such as `standard`, `mature`, or `research`, define the precise permissions, jurisdictional flags, and required proofs that govern an artifact's lifecycle . For example, a shard with `entitlement_profile_id: 'research-neural-lab-v1'` would be understood by all components of the constellations to require additional ethical and BCI-related proofs before it can be activated . This field works in concert with the `prooftype` field, which specifies the kind of ZKP (Zero-Knowledge Proof) that must be submitted to verify compliance with the profile's requirements (e.g., `bcistate`, `age_gating`, `charter_agreement`) . When a runtime or tool receives a shard, it can consult its local registry of verifier contracts to determine how to validate the proof blob associated with that `prooftype`, all routed through the `verifier_id` specified in the `zkpproofv1` envelope . This creates a tightly coupled, auditable chain of command where the shard itself declares not only its own properties but also the exact cryptographic and policy checks it must pass to be considered valid.

The `deadledgerref` field acts as the ultimate anchor, creating an opaque but verifiable link between a Tier 2 artifact (like a bundle or persona) and its official attestation in the Dead-Ledger . The lifecycle of this reference is strictly defined: a vault repository computes the hash of an artifact, registers it with Dead-Ledger, which then generates a `deadledgerref` and returns it to the vault . The vault then writes this

deadledgerref into its own registries and into the QPUdatashard it produces . From that point forward, any request to load or use that artifact must be accompanied by its deadledgerref . The runtime's Policy client then uses this reference to query Dead-Ledger for the full attestation entry, which contains the definitive safetytier, intensityband, invariant_ranges, and required proofs for that specific artifact . This mechanism prevents any "backdoor" access to high-risk content and ensures that all entitlement decisions are made by the centralized, auditable authority of Dead-Ledger, not by individual vaults or runtimes . The QPUdatashard thus becomes a portable container for this critical piece of evidence—the deadledgerref.

This explicit linkage extends to the BCI swing functions, which are perhaps the most dynamic examples of the system's governance model. Each bci_swing_* shard must declare its safetytier, entitlement_profile_id, and prooftype . When a BCI system detects a signal that should trigger a swing, it doesn't execute the transformation directly. Instead, it constructs a zkpproofv1 envelope, similar to those used for age gating or charter agreements, proving that the current BCI state satisfies the conditions required to invoke that specific swing . For example, to trigger bci_swing_uec_soften_01, the system would generate a bcistate proof stating "current fear_index > threshold AND overload_flag is true." This proof is then submitted to Dead-Ledger along with the deadledgerref of the active session or bundle. Dead-Ledger's verifier validates the proof and, if successful, returns an entitlement decision token that authorizes the engine to apply the UEC and intensity_band adjustments specified by the swing . This process ensures that even the most rapid, real-time adaptations driven by BCI data are subject to the same rigorous, cryptographic, and policy-based scrutiny as any other access control decision. It transforms the BCI from a raw sensor into a provable input for a formal, trustless adjudication process, fully realizing the vision of a proof-centric, schema-driven ecosystem.

NDJSON as the Wire Protocol for Orchestrated Data Exchange

The choice of Newline Delimited JSON (NDJSON) as the serialization format for QPUdatashards is a pragmatic and powerful decision that underpins the entire contract layer's effectiveness . NDJSON, a format where each line in a file is a separate, self-contained JSON object, is exceptionally well-suited to the streaming, distributed nature of the Horror$Place VM-constellation . Its line-oriented structure provides a natural and

efficient way to represent a sequence of independent `QPUdatashards`, making it ideal for logs, telemetry feeds, and data pipelines that move artifacts between repositories . Unlike a single, monolithic JSON array, which requires parsing the entire file to find a specific entry, NDJSON allows for easy, head-tail, or random-access processing of individual shards without loading the whole dataset into memory. This efficiency is critical for high-throughput scenarios where thousands or millions of shards may be generated and exchanged during a single session or development cycle.

The primary advantage of NDJSON in this context is its simplicity and universality, which directly facilitates the goal of language-agnosticism . Every programming language with standard JSON support—be it Rust, Lua, Python, or PowerShell—is equipped to read and write NDJSON with minimal effort . An AI generator can simply be instructed to "emit a shard as an NDJSON line," and the target language's standard libraries will handle the rest. This avoids the complexities and potential inconsistencies of binary formats or more verbose text-based formats like XML. Each line in an NDJSON stream is a valid JSON object, which means it can be trivially converted into native data structures like Rust structs, Lua tables, or Python dictionaries . This seamless conversion acts as the "prism" that refracts the uniform `QPUdatashard` contract into language-specific representations, enabling a developer or AI agent to work with strongly-typed, structured data in their preferred environment while still adhering to a common, platform-independent wire format . This simplicity is the key to making the contract universally executable by any tool capable of emitting text.

Furthermore, the inherent property of NDJSON—that each line is a standalone record—is perfectly aligned with the requirement for self-contained shards destined for cross-repo use . As stipulated in the `QPUdatashard Vocabulary v1` document, any shard crossing a repository or tier boundary must contain all necessary fields—including `schemaref`, `repo_tier`, `deadledgerref`, `invariants`, `metrics`, and `function_kind`—within that single line . This design choice eliminates fragile dependencies on external manifests, parent-child relationships, or context inherited from previous lines. It allows for maximum resilience and composability. A CI pipeline, an orchestrator, or a verifier service can validate and process each shard in isolation, knowing that it contains all the information it needs to make a decision . This is particularly important in a distributed system where components may be updated or replaced independently. For instance, a shard can be passed from a Tier 2 vault to a Tier 1 registry and then to a Tier 3 verifier without losing its provenance or context. While sparse or context-inheriting shards may be acceptable for local, intra-repo logging within a single repository, they are explicitly discouraged for any flow that crosses boundaries, and NDJSON's structure naturally supports this distinction . The format encourages the

production of "fat" and durable artifacts that can withstand the rigors of a complex, multi-party ecosystem.

The adoption of NDJSON also has practical implications for tooling and observability. Standard command-line utilities like `jq`, `awk`, or `sed` can be readily used to filter, transform, and analyze streams of `QPUdatashards`, enabling quick debugging and ad-hoc analysis [6]. Logs written in NDJSON are human-readable and machine-parsable, providing a clear audit trail of all significant events and artifacts flowing through the system. This is invaluable for security audits, performance profiling, and understanding the lineage of generated content. The format's compatibility with existing data processing ecosystems means that these shard streams can be ingested into larger data lakes or analytics platforms for long-term trend analysis, such as correlating BCI swing triggers with changes in `UEC` and `ARR` metrics across thousands of sessions. The choice of NDJSON is therefore not merely a technical detail but a strategic one that enables the core objectives of the research: creating a robust, scalable, and interoperable contract layer that can be reliably implemented by diverse teams and automated systems across the entire Horror$Place constellation.

Operationalizing the Contract with AI and CI Tooling

The `QPUdatashard` vocabulary and NDJSON contract layer are not merely theoretical abstractions; their ultimate purpose is to be operationalized by the very tools that drive development within the Horror$Place ecosystem, namely AI assistants like GitHub Copilot and automated CI/CD pipelines . The design of the contract is explicitly tailored to be directly consumed and enforced by these systems, transforming high-level doctrinal goals into concrete, machine-executable instructions. For AI tools, the contract provides a clear and unambiguous grammar for generating code. An instruction such as "In `Horror.Place`, generate a Rust module for a persona implementation that routes prompts based on invariants and metrics" can now be translated into a precise directive: "Emit a `persona_impl_shard` as an NDJSON line. Set `schemaref` to `schema:Horror.Place/core/schemas/personacontractv1.json`, populate the `invariants` and `metrics` objects with the appropriate bands, and include a placeholder for `deadledgerref`" . The AI agent, armed with the vocabulary document, can reliably produce the correct output in Rust, Lua, or YAML, as the target is the NDJSON structure itself, not the final compiled artifact . This drastically reduces ambiguity and ensures a high degree of consistency and correctness across all generated code.

The integration with CI/CD pipelines is equally critical for enforcing the contract at scale. The CI workflows in each repository must be configured to act as validators of the `QPUdatashard` contract . This involves implementing a series of checks that run on every pull request or merge. First, the CI system should validate that any generated NDJSON files conform to the common envelope structure and that all required fields are present . Second, it must perform schema validation, ensuring that the payload of each shard matches the schema referenced by its `schemaref` field . This prevents developers from accidentally deviating from the canonical definitions. Third, the CI should verify that cross-repo references are handled correctly. For example, it can check that any shard produced in a Tier 2 vault contains a valid `deadledgerref` placeholder and that the `schemaref` points to a schema within the permitted Tier 1 or Tier 3 repositories . Negative tests should also be included in the CI suite to confirm that out-of-range invariants or malformed proofs are rejected according to the policy profiles, ensuring that the system's guardrails are functioning as intended . By automating these checks, the contract becomes a continuously enforced rule, not just static documentation, fostering a culture of compliance and reducing the risk of introducing vulnerabilities or inconsistencies.

For IDE integrations and AI chat interfaces, the `.aln` configuration file for the repository becomes the central hub for enforcing these rules . This file can encode directives that tell the AI agent which shard kinds are permissible, what the allowed `function_kinds` are, and which schemas can be referenced . It can enforce constraints such as treating `deadledgerref` as mandatory for certain artifact types and prohibiting the emission of real proof blobs, BCI samples, or any other raw content . The AI would be constrained to generate only `zkpproofv1` envelopes, which contain placeholders for the `proof_blob` and commit to only encoding non-sensitive `public_inputs` . This tight integration ensures that the AI assistant acts as a compliant citizen of the VM-constellation, always operating within the prescribed boundaries of the proof- and schema-centric doctrine. The `.aln` file essentially codifies the governance rules, making them an integral part of the development workflow itself . This comprehensive approach, combining explicit AI guidance, automated CI validation, and IDE-level enforcement, ensures that the `QPUdatashard` contract is not just a blueprint but a living, breathing standard that shapes and improves the quality of every artifact generated across the entire ecosystem.

Reference

1. [PDF] Oracle PaaS and IaaS Universal Credits Service Descriptions https://www.oracle.com/asean/contracts/docs/paas_iaas_universal_credits_3940775.pdf?download=false
2. [PDF] un-transparency-protocol.pdf https://untp.unece.org/un-transparency-protocol.pdf
3. Taskpane Dict | PDF - Scribd https://www.scribd.com/document/615646225/Taskpane-Dict
4. 6 - CodaLab Worksheets https://worksheets.codalab.org/rest/bundles/0xadf98bb30a99476ab56ebff3e462d4fa/contents/blob/glove.6B.100d.txt-vocab.txt
5. What is GitHub's character limit, or line length for viewing files on ... https://stackoverflow.com/questions/22207920/what-is-githubs-character-limit-or-line-length-for-viewing-files-on-github
6. Reconciling Copilot usage metrics across dashboards, APIs, and ... https://docs.github.com/en/copilot/reference/copilot-usage-metrics/reconciling-usage-metrics
7. How Claude and GitHub Copilot search for code differently - LinkedIn https://www.linkedin.com/posts/stride-build_claude-code-vs-github-copilot-the-semantic-activity-7343819584937803777-ra2B
8. GitHub Copilot usage metrics https://docs.github.com/en/copilot/concepts/copilot-usage-metrics/copilot-metrics
9. Lecture Notes in Networks and Systems 1523 - Springer Nature https://link.springer.com/content/pdf/10.1007/978-981-96-9935-3.pdf
10. Copilot CLI ACP server - GitHub Enterprise Cloud Docs https://docs.github.com/en/enterprise-cloud@latest/copilot/reference/copilot-cli-reference/acp-server
11. Proceedings of the 19th International Conference on Hybrid Systems https://dl.acm.org/doi/proceedings/10.1145/2883817?tocHeading=heading1
12. [PDF] Brain-Computer Interfaces: Applying our Minds to Human-Computer ... https://www.researchgate.net/profile/Christa-Neuper-2/publication/226311836_Could_anyone_use_a_BCI/links/00463532c195016891000000/Could-anyone-use-a-BCI.pdf
13. Cognitive Sensors, Volume 2 - IOP Science https://iopscience.iop.org/book/edit/978-0-7503-5346-5.preview.pdf

14. General Ledger Account Hierarchy - SAP Help Portal https://help.sap.com/docs/SAP_S4HANA_ON-PREMISE/ee6ff9b281d8448f96b4fe6c89f2bdc8/9496222dee2e41e48e663914fdd128ab.html
15. [PDF] A guide to auditing trust accounts - Consumer Protection https://www.consumerprotection.wa.gov.au/system/files/documents/2025-01/sareauditingguide_3.pdf
16. Exploring vision transformers for deep feature extraction ... - Nature https://www.nature.com/articles/s41598-026-45087-y_reference.pdf
17. EEG-Based User Identification in Gaming: Comparative Analysis of ... https://dl.acm.org/doi/full/10.1145/3709026.3709084
18. Contemporary Chinese Horror Films: Genre, Censorship, Market https://escholarship.org/uc/item/1tk37839
19. Multichannel Audio - an overview | ScienceDirect Topics https://www.sciencedirect.com/topics/computer-science/multichannel-audio
20. CDS Views for Stock and Material Documents - SAP Help Portal https://help.sap.com/docs/SAP_S4HANA_ON-PREMISE/ee6ff9b281d8448f96b4fe6c89f2bdc8/3f9a540204ec433a9c793e86d7dc7a96.html
21. [PDF] GUIDELINES RESULTING FROM MEETINGS OF THE VAT ... https://taxation-customs.ec.europa.eu/system/files/2024-01/guidelines-vat-committee-meetings_en.pdf
22. Fiscal Year Variant of Ledger Company Code - SAP Help Portal https://help.sap.com/docs/SAP_S4HANA_ON-PREMISE/ee6ff9b281d8448f96b4fe6c89f2bdc8/5c290d233dd942d79ce91cce4ba6498f.html
23. [PDF] Consolidated text of the Common Reporting Standard (2025) | OECD https://www.oecd.org/content/dam/oecd/en/publications/reports/2025/04/consolidated-text-of-the-common-reporting-standard-2025_e478bc04/055664b1-en.pdf
24. [PDF] ESMA74-449133380-425 Consultation Paper on the Technical ... https://www.esma.europa.eu/sites/default/files/2023-07/ESMA74-449133380-425_MiCA_Consultation_Paper_1st_package.pdf
25. W3C Alternative and Augmented Communication (AAC) Symbol ... https://www.w3.org/TR/aac-registry/
26. Brain-Inspired Cognitive Architectures for Artificial Intelligence: BICA ... https://link.springer.com/content/pdf/10.1007/978-3-030-65596-9.pdf
27. Experts for Nodejs-Mobile-React-Native Plugins Readme - Linknovate https://www.linknovate.com/search/?query=nodejs-mobile-react-native%20plugins%20readme
28. Experts for Nice Vr-Native Reload Style - Linknovate https://www.linknovate.com/search/?query=nice%20vr-native%20reload%20style

29. How to handle wide markdown tables and line-length checks in pre ... https://stackoverflow.com/questions/74319856/how-to-handle-wide-markdown-tables-and-line-length-checks-in-pre-commit
30. Intel® VTune™ Profiler Release Notes and New Features https://www.intel.com/content/www/us/en/developer/articles/release-notes/vtune-profiler/current.html
31. [PDF] HDAWG User Manual - Zurich Instruments Documentation https://docs.zhinst.com/pdf/ziHDAWG_UserManual.pdf
32. [PDF] Documentation 3.0 - Zabbix https://www.zabbix.com/documentation/3.0/downloads/Zabbix_Documentation_3.0.en.pdf
33. [PDF] FX Series RFID Fixed Reader Integration Guide (en) https://www.zebra.com/content/dam/support-dam/en/documentation/unrestricted/guide/product/fxseries-ig-en.pdf
34. Best practices for using GitHub Copilot https://docs.github.com/en/copilot/get-started/best-practices
35. CCS '22: Proceedings of the 2022 ACM SIGSAC Conference on ... https://dl.acm.org/doi/proceedings/10.1145/3548606?tocHeading=heading2
36. Web Information Systems Engineering – WISE 2024 - Springer Nature https://link.springer.com/content/pdf/10.1007/978-981-96-0573-6.pdf
37. An extensive review of tools for manual annotation of documents https://academic.oup.com/bib/article/22/1/146/5670958